Sensor Fusion & Association

ELG 5218 - Uncertainty Evaluation in Engineering Measurements and Machine Learning

Miodrag Bolic

University of Ottawa

March 11, 2026

Lecture Roadmap

Block 1 — Optimal Gaussian Multi-Sensor Fusion

- MLE derivation of precision-weighted fusion
- Entropy and information view
- Multivariate case; N sensors
- Correlated noise and its consequences

Block 2 — Bayesian Fusion

- Kalman Filter as Bayesian fusion
- Sequential vs. stacked updates
- Information form: precision additivity
- Covariance Intersection (CI)

Block 3 — Data Association

- Gating; Global Nearest Neighbour
- PDAF: soft association, covariance spread term
- JPDA: joint events, marginal probabilities
- MHT: deferred decisions

Block 4 — Registration and System Integration

- Sensor registration: spatial and temporal alignment
- Track-to-track fusion
- PHD filter overview
- Complete multi-sensor pipeline

The running question

"Multiple sensors produce noisy measurements of the same environment. How can we combine these measurements to estimate the state of the world, and how do we handle situations where it is unclear which measurement corresponds to which object?"

Notebook: notebook_traditional_fusion.ipynb

Block 1

Optimal Multi-Sensor Fusion for Gaussian Random Variables

The Fusion Problem: Setup

Two sensors measure the **same scalar quantity** x:

$$y_1 = z + v_1, \quad v_1 \sim \mathcal{N}(0, \sigma_1^2)$$

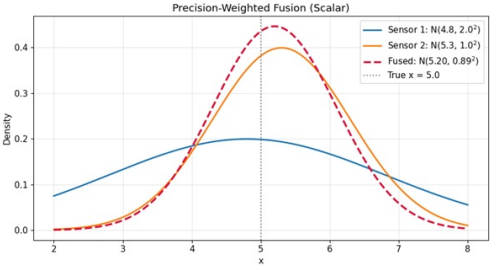
$$y_2 = z + v_2, \quad v_2 \sim \mathcal{N}(0, \sigma_2^2)$$

Noise is independent: $\mathbb{E}[v_1 v_2] = 0$.

Question: given y_1 and y_2, what is the best single estimate $\hat{x}$?

Best

Among all linear unbiased estimators $\hat{z} = w_1 y_1 + w_2 y_2$, minimise $\mathrm{Var}[\hat{z}]$ subject to $w_1 + w_2 = 1$.



Derivation: Maximum Likelihood

Joint likelihood (independent noise):

$$\log p(y_1, y_2 \mid z) = -\frac{(y_1 - z)^2}{2\sigma_1^2} - \frac{(y_2 - z)^2}{2\sigma_2^2} + \text{const}$$

Set derivative to zero:

$$\frac{\partial}{\partial z} \log p = \frac{y_1 - z}{\sigma_1^2} + \frac{y_2 - z}{\sigma_2^2} = 0$$

$$\Rightarrow \quad z\left(\frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}\right) = \frac{y_1}{\sigma_1^2} + \frac{y_2}{\sigma_2^2}$$

Precision-weighted fusion (scalar)

Define precisions $\lambda_i = 1/\sigma_i^2$. Then:

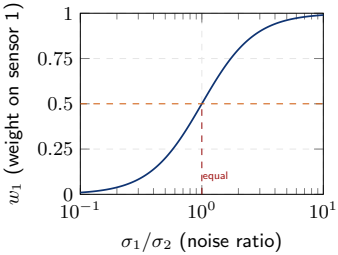
$$\boxed{\hat{z} = \frac{\lambda_1 y_1 + \lambda_2 y_2}{\lambda_1 + \lambda_2}} \qquad \boxed{\sigma_f^{-2} = \lambda_1 + \lambda_2}$$

Verify: $\hat{x}$ is unbiased; $\sigma_f^2 < \min(\sigma_1^2, \sigma_2^2)$ always.
Equivalently: $w_i = \lambda_i/(\lambda_1 + \lambda_2)$, so $w_1 + w_2 = 1$. ✓

Fusion Weights as Soft Exclusion

The weight assigned to sensor 1:

$$w_1 = \frac{\sigma_2^2}{\sigma_1^2 + \sigma_2^2} = \frac{1}{1 + (\sigma_2/\sigma_1)^2}$$



Three regimes:

- $\sigma_1 = \sigma_2$: equal weights, $w_1 = 0.5$
- $\sigma_1 \gg \sigma_2$: $w_1 \to 0$ (ignore noisy sensor)
- $\sigma_1 \ll \sigma_2$: $w_1 \to 1$ (rely on precise sensor)

Automatic soft exclusion

No thresholds, no if-statements. A degraded sensor gets near-zero weight automatically. This is precision-weighting doing its job.

Fusion Reduces Entropy (Gaussian Case)

Entropy of a 1D Gaussian: If $x \sim \mathcal{N}(\mu, \sigma^2)$ then

$$H(x) \;=\; \tfrac{1}{2}\log\!\big(2\pi e\,\sigma^2\big) \quad \text{(nats)}.$$

Since H is monotone in σ^2, *lower variance* $\Rightarrow$ *lower entropy*.

After Sensor 1: Let the posterior (given Sensor 1) be $z \mid y_1 \sim \mathcal{N}(\mu,\, \sigma_1^2)$.

Add Sensor 2 (independent, Gaussian): Let Sensor 2 have measurement noise variance σ_2^2 (precision $\lambda_2 = \sigma_2^{-2}$). Then the fused posterior has variance

$$\sigma_f^2 \;=\; \left(\tfrac{1}{\sigma_1^2} + \tfrac{1}{\sigma_2^2}\right)^{-1} \;=\; (\lambda_1 + \lambda_2)^{-1}, \quad \text{where } \lambda_1 = \sigma_1^{-2}.$$

Information gain (entropy drop) from Sensor 2:

$$\Delta I \;=\; H(z \mid y_1) - H(z \mid y_1, y_2) \;=\; \tfrac{1}{2}\log\frac{\sigma_1^2}{\sigma_f^2} \;=\; \tfrac{1}{2}\log\!\Big(1 + \frac{\lambda_2}{\lambda_1}\Big) \;\geq\; 0.$$

Takeaway: Fusion always reduces entropy

Because $\lambda_f = \lambda_1 + \lambda_2 \geq \lambda_1$, we have $\sigma_f^2 \leq \sigma_1^2$, hence $H(z \mid y_1, y_2) \leq H(z \mid y_1)$. Equality holds only when Sensor 2 is uninformative ($\sigma_2^2 \to \infty$).

Direct Effect on Error Bars (1D) + Quick Example

From information gain (bits) to error bars:

$$\Delta I_{\text{bits}} = \frac{1}{2}\log_2 \frac{\sigma_1^2}{\sigma_f^2} \quad \Longrightarrow \quad \frac{\sigma_f^2}{\sigma_1^2} = 2^{-2\Delta I_{\text{bits}}}, \qquad \frac{\sigma_f}{\sigma_1} = 2^{-\Delta I_{\text{bits}}}.$$

Interpretation:

- Variance shrinks by $2^{2\Delta I_{\text{bits}}}$.
- RMSE (std. dev.) *and* 95% CI width shrink by $2^{\Delta I_{\text{bits}}}$.

Quick numerical intuition

After Sensor 1: $\sigma_1^2 = 1.0$.
Sensor 2: $\sigma_2^2 = 0.25 \Rightarrow \lambda_2 = 4$.

Fuse: $\sigma_f^2 = (1+4)^{-1} = 0.2$.

Information gain:
$\Delta I = \frac{1}{2}\ln(1/0.2) = \frac{1}{2}\ln 5 \approx 0.8047$ nats ≈ 1.16 bits.

Error bars: $\sigma_f/\sigma_1 = 2^{-1.16} \approx 1/2.24$ ($\sim 2.2\times$ tighter).

Concrete units (e.g. position)

If $\sigma_1 = 3\,\text{m}$ after Sensor 1, then
$\sigma_f \approx 3/2.22 \approx 1.35\,\text{m}$.

Multivariate Fusion

State $\boldsymbol{z} \in \mathbb{R}^n$; sensors measure $\boldsymbol{y}_i = \boldsymbol{H}_i \boldsymbol{z} + \boldsymbol{r}_i$, $\boldsymbol{r}_i \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{R}_i)$, independent.
Joint log-likelihood:

$$\log p(\boldsymbol{y}_1, \boldsymbol{y}_2 \mid \boldsymbol{z}) = -\tfrac{1}{2}(\boldsymbol{y}_1 - \boldsymbol{H}_1 \boldsymbol{z})^\top \boldsymbol{R}_1^{-1}(\boldsymbol{y}_1 - \boldsymbol{H}_1 \boldsymbol{z}) - \tfrac{1}{2}(\boldsymbol{y}_2 - \boldsymbol{H}_2 \boldsymbol{z})^\top \boldsymbol{R}_2^{-1}(\boldsymbol{y}_2 - \boldsymbol{H}_2 \boldsymbol{z}) + \text{const}$$

Setting gradient to zero and solving gives the fused Gaussian:

$$\boldsymbol{\Sigma}_f^{-1} = \boldsymbol{H}_1^\top \boldsymbol{R}_1^{-1} \boldsymbol{H}_1 + \boldsymbol{H}_2^\top \boldsymbol{R}_2^{-1} \boldsymbol{H}_2$$

$$\boldsymbol{\mu}_f = \boldsymbol{\Sigma}_f \left(\boldsymbol{H}_1^\top \boldsymbol{R}_1^{-1} \boldsymbol{y}_1 + \boldsymbol{H}_2^\top \boldsymbol{R}_2^{-1} \boldsymbol{y}_2 \right)$$

For $\boldsymbol{H}_i = \boldsymbol{I}$ (both sensors measure the full state directly), with $\boldsymbol{y}_i = \boldsymbol{z} + \boldsymbol{r}_i$ and $\boldsymbol{r}_i \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{\Sigma}_i)$:

$$\boxed{\boldsymbol{\Sigma}_f^{-1} = \boldsymbol{\Sigma}_1^{-1} + \boldsymbol{\Sigma}_2^{-1}} \qquad \boxed{\boldsymbol{\mu}_f = \boldsymbol{\Sigma}_f \left(\boldsymbol{\Sigma}_1^{-1} \boldsymbol{y}_1 + \boldsymbol{\Sigma}_2^{-1} \boldsymbol{y}_2 \right)}$$

Entropy decreases: $\log |\boldsymbol{\Sigma}_f| < \min(\log |\boldsymbol{\Sigma}_1|, \log |\boldsymbol{\Sigma}_2|)$ (positive-definite ordering).

This is the KF information form

The information filter propagates the precision $\boldsymbol{\Lambda} = \boldsymbol{\Sigma}^{-1}$ (and information vector $\boldsymbol{\eta} = \boldsymbol{\Lambda}\boldsymbol{\mu}$), adding *measurement precision* $\boldsymbol{H}^\top \boldsymbol{R}^{-1} \boldsymbol{H}$ each time a sensor reports.

N Sensors and the $1/\sqrt{N}$ Law

For N sensors measuring the same quantity with variances $\sigma_1^2, \ldots, \sigma_N^2$:

$$\sigma_f^{-2} = \sum_{i=1}^{N} \sigma_i^{-2} \qquad \hat{z} = \sigma_f^2 \sum_{i=1}^{N} \frac{y_i}{\sigma_i^2}$$

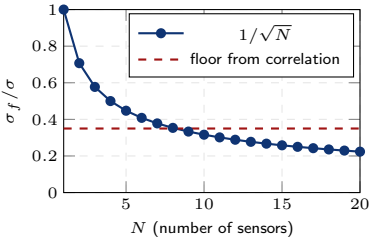
Special case — identical sensors $\sigma_i = \sigma$:

$$\hat{z} = \frac{1}{N} \sum_{i=1}^{N} y_i, \qquad \sigma_f = \frac{\sigma}{\sqrt{N}}$$

The simple average is optimal. Variance shrinks as $1/N$.

What breaks this ideal law?

1. **Bias:** if $\mathbb{E}[v_i] \neq 0$, reducing variance does not help. Mean error stays. All N sensors share the same bias.
2. **Correlation:** if $\mathrm{Cov}(v_i, v_j) \neq 0$, variance reduction saturates.
3. **Unknown association:** if you don't know y_i came from the target, the formula cannot be applied.



Correlated Sensor Noise: When Precision Additivity Fails

Same setup: $y_i = z + v_i$, but now the noise terms share a common component:

$$v_1 = u + e_1, \quad v_2 = u + e_2, \qquad u \sim \mathcal{N}(0, \sigma_u^2), \;\; e_i \sim \mathcal{N}(0, \sigma_{e,i}^2), \;\; u \perp e_1 \perp e_2$$

Each sensor keeps its own total variance from before, $\sigma_1^2 = \sigma_u^2 + \sigma_{e,1}^2$, $\sigma_2^2 = \sigma_u^2 + \sigma_{e,2}^2$, but now $\mathrm{Cov}(v_1, v_2) = \sigma_u^2 \neq 0$.

The joint noise covariance becomes:

$$\mathbf{\Sigma} = \begin{bmatrix} \sigma_1^2 & \sigma_u^2 \\ \sigma_u^2 & \sigma_2^2 \end{bmatrix}$$

The correct fused variance (via $\sigma_f^{-2} = \mathbf{1}^\top \mathbf{\Sigma}^{-1} \mathbf{1}$):

$$\sigma_f^2 = \frac{\sigma_1^2 \, \sigma_2^2 - \sigma_u^4}{\sigma_1^2 + \sigma_2^2 - 2\,\sigma_u^2}$$

What happens if we ignore σ_u^2 and use the naive formula?

$$\hat{\sigma}_{f,\text{naive}}^2 = \frac{\sigma_1^2 \, \sigma_2^2}{\sigma_1^2 + \sigma_2^2} \;<\; \sigma_f^2 \quad \text{when } \sigma_u^2 > 0$$

Overconfidence from ignored correlation

Naive fusion underestimates the fused uncertainty. The filter *believes* it is more accurate than it actually is, leading to track divergence in the KF. Covariance Intersection (Block 2) gives a principled remedy.

Analytical Example — Block 1

Two sensors: work it by hand

$\sigma_1 = 2,\ \ \sigma_2 = 1,$ true $z = 5,$ observed $y_1 = 4.8,\ \ y_2 = 5.3.$

Step 1 — Precisions: $\lambda_1 = 1/4 = 0.25, \quad \lambda_2 = 1/1 = 1.0, \quad \lambda_f = 1.25.$
Step 2 — Weights: $w_1 = 0.25/1.25 = 0.20, \quad w_2 = 1.0/1.25 = 0.80.$
Step 3 — Fused estimate:

$$\hat{z} = 0.20 \times 4.8 + 0.80 \times 5.3 = 0.96 + 4.24 = \mathbf{5.20}$$

Step 4 — Fused variance: $\sigma_f^2 = 1/1.25 = 0.80, \quad \sigma_f = 0.894.$
Check: $\sigma_f = 0.89 < \sigma_2 = 1 < \sigma_1 = 2.$ ✓

Errors: sensor 1: $|4.8 - 5| = 0.2$; sensor 2: $|5.3 - 5| = 0.3$; fused: $|5.2 - 5| = 0.2$.
On average (Monte Carlo): fused RMSE $\approx \sigma_f = 0.89$ vs. $\sigma_2 = 1$ for the better sensor alone.

Block 2

Bayesian Fusion

From Kalman Filters to Deep Probabilistic Models

Kalman Filter as Bayesian Fusion

State (dynamics) model (Gaussian):

$$\boldsymbol{z}_t = \boldsymbol{F}_t \boldsymbol{z}_{t-1} + \boldsymbol{q}_t, \qquad \boldsymbol{q}_t \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{Q}_t)$$

Observation models for modalities $m = 1, \ldots, M$:

$$\boldsymbol{y}_t^{(m)} = \boldsymbol{H}_t^{(m)} \boldsymbol{z}_t + \boldsymbol{r}_t^{(m)}, \qquad \boldsymbol{r}_t^{(m)} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{R}_t^{(m)})$$

Predict step:

$$\boldsymbol{\mu}_{t|t-1} = \boldsymbol{F}_t \, \boldsymbol{\mu}_{t-1|t-1}, \qquad \boldsymbol{\Sigma}_{t|t-1} = \boldsymbol{F}_t \, \boldsymbol{\Sigma}_{t-1|t-1} \boldsymbol{F}_t^\top + \boldsymbol{Q}_t$$

Update step (single modality m shown):

$$\hat{\boldsymbol{y}}_t^{(m)} = \boldsymbol{H}_t^{(m)} \boldsymbol{\mu}_{t|t-1}, \qquad \boldsymbol{S}_t^{(m)} = \boldsymbol{H}_t^{(m)} \boldsymbol{\Sigma}_{t|t-1} \left(\boldsymbol{H}_t^{(m)} \right)^\top + \boldsymbol{R}_t^{(m)}$$

$$\boldsymbol{K}_t^{(m)} = \boldsymbol{\Sigma}_{t|t-1} \left(\boldsymbol{H}_t^{(m)} \right)^\top \left(\boldsymbol{S}_t^{(m)} \right)^{-1}$$

$$\boldsymbol{\mu}_{t|t} = \boldsymbol{\mu}_{t|t-1} + \boldsymbol{K}_t^{(m)} \left(\boldsymbol{y}_t^{(m)} - \hat{\boldsymbol{y}}_t^{(m)} \right), \qquad \boldsymbol{\Sigma}_{t|t} = \left(\boldsymbol{I} - \boldsymbol{K}_t^{(m)} \boldsymbol{H}_t^{(m)} \right) \boldsymbol{\Sigma}_{t|t-1}$$

Uncertainty state

Covariance $\boldsymbol{\Sigma}_{t|t}$ is the **uncertainty state** — it propagates through every predict/update cycle.

Multi-Sensor KF: Sequential vs. Stacked Updates

Scenario: velocity IMU at 100 Hz + GPS at 1 Hz.

Sequential (iterated) update:
Apply each modality's update in turn, starting from the prior $(\boldsymbol{\mu}_{t|t-1}, \boldsymbol{\Sigma}_{t|t-1})$:

$$\boldsymbol{\Sigma}_{t|t}^{(0)} := \boldsymbol{\Sigma}_{t|t-1}, \qquad \boldsymbol{\Sigma}_{t|t}^{(m)} = \left(\boldsymbol{I} - \boldsymbol{K}_t^{(m)} \boldsymbol{H}_t^{(m)}\right) \boldsymbol{\Sigma}_{t|t}^{(m-1)}$$

- Exact when modality noises are independent
- Order does not matter under independence

Joint (stacked) update:

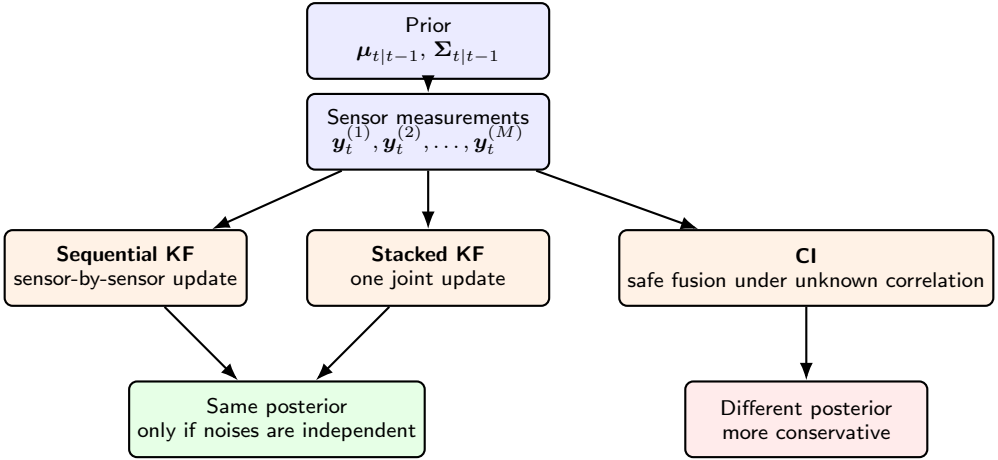
$$\boldsymbol{y}_t = \begin{pmatrix} \boldsymbol{y}_t^{(1)} \\ \boldsymbol{y}_t^{(2)} \end{pmatrix}, \quad \boldsymbol{H}_t = \begin{pmatrix} \boldsymbol{H}_t^{(1)} \\ \boldsymbol{H}_t^{(2)} \end{pmatrix}, \quad \boldsymbol{R}_t = \begin{pmatrix} \boldsymbol{R}_t^{(1)} & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{R}_t^{(2)} \end{pmatrix}$$

- One matrix inversion, same result
- Fails if $\boldsymbol{R}_t$ has off-diagonal terms

Engineering hazard

Off-diagonal terms in $\boldsymbol{R}_t$ (correlated sensor noise) cause overconfident fusion if set to zero. This motivates Covariance Intersection.

How Do We Fuse Multiple Sensors?



Good for asynchronous sensors

Convenient for synchronous fusion

Protects against overconfident fusion

KF Multi-Sensor Update: Worked Example

1-D position–velocity state

State $\boldsymbol{z}_t = [\,p,\ v\,]^\top$.
Predicted (from the *Prediction Step*):

$$\boldsymbol{\mu}_{t|t-1} = \begin{pmatrix} 10 \\ 2 \end{pmatrix}, \qquad \boldsymbol{\Sigma}_{t|t-1} = \begin{pmatrix} 5 & 1 \\ 1 & 2 \end{pmatrix}.$$

GPS measures position:

$$\mathbf{H}_t = \begin{bmatrix} 1 & 0 \end{bmatrix}, \qquad \mathbf{R}_t = 4, \qquad \mathbf{y}_t = 12.$$

Step 1 — Predicted measurement: $\hat{\mathbf{y}}_t = \mathbf{H}_t\,\boldsymbol{\mu}_{t|t-1} = 10.$
Step 2 — Innovation (residual): $\tilde{\mathbf{y}}_t = \mathbf{y}_t - \hat{\mathbf{y}}_t = 12 - 10 = 2.$
Step 3 — Innovation covariance: $\mathbf{S}_t = \mathbf{H}_t\,\boldsymbol{\Sigma}_{t|t-1}\,\mathbf{H}_t^\top + \mathbf{R}_t = 5 + 4 = 9.$
Step 4 — Kalman gain: $\mathbf{K}_t = \boldsymbol{\Sigma}_{t|t-1}\mathbf{H}_t^\top\mathbf{S}_t^{-1} = \begin{pmatrix} 5 \\ 1 \end{pmatrix} \cdot \frac{1}{9} = \begin{pmatrix} 0.556 \\ 0.111 \end{pmatrix}.$
Step 5 — Updated mean: $\boldsymbol{\mu}_{t|t} = \boldsymbol{\mu}_{t|t-1} + \mathbf{K}_t\,\tilde{\mathbf{y}}_t = \begin{pmatrix} 10 \\ 2 \end{pmatrix} + \begin{pmatrix} 0.556 \\ 0.111 \end{pmatrix} \cdot 2 = \begin{pmatrix} 11.11 \\ 2.22 \end{pmatrix}.$
Step 6 — Updated covariance:

$$\boldsymbol{\Sigma}_{t|t} = (\mathbf{I} - \mathbf{K}_t\mathbf{H}_t)\,\boldsymbol{\Sigma}_{t|t-1} = \begin{pmatrix} 2.22 & 0.44 \\ 0.44 & 1.89 \end{pmatrix}.$$

Check (uncertainty reduced): $\det(\boldsymbol{\Sigma}_{t|t}) \approx 4.00 \;<\; \det(\boldsymbol{\Sigma}_{t|t-1}) = 9.$ ✓

Covariance Intersection (CI) — The Problem

Why Standard Fusion Fails

Two *unbiased* Gaussian estimates of the *same* state:

$$(\boldsymbol{\mu}_1, \boldsymbol{\Sigma}_1) \quad \text{and} \quad (\boldsymbol{\mu}_2, \boldsymbol{\Sigma}_2)$$

Their joint error covariance requires the cross-term $\boldsymbol{\Sigma}_{12}$:

$$\boldsymbol{\Sigma}_{\text{joint}} = \begin{bmatrix} \boldsymbol{\Sigma}_1 & \boldsymbol{\Sigma}_{12} \\ \boldsymbol{\Sigma}_{12}^\top & \boldsymbol{\Sigma}_2 \end{bmatrix} \succeq 0$$

but $\boldsymbol{\Sigma}_{12}$ is **unknown** (shared data ancestry, network loops, ...).

Tempting (but wrong) fix:

Set $\boldsymbol{\Sigma}_{12} = \mathbf{0}$ and apply standard Kalman fusion.

$\Rightarrow$ double-counts shared information

$\Rightarrow$ $\boldsymbol{\Sigma}_f$ **underestimates** true uncertainty

$\Rightarrow$ **overconfident, inconsistent estimate**

What we need instead

A fused estimate $(\boldsymbol{\mu}_f, \boldsymbol{\Sigma}_f)$ that is:

- **Unbiased**: $\omega_1 + \omega_2 = \mathbf{I}$
- **Consistent**: $\boldsymbol{\Sigma}_f$ upper-bounds the true posterior covariance *for any* admissible $\boldsymbol{\Sigma}_{12}$
- **As tight as possible**

Covariance Intersection (CI) — Fusion Rule & Algorithm

CI fusion rule (two Gaussians $(\boldsymbol{\mu}_1, \boldsymbol{\Sigma}_1)$ and $(\boldsymbol{\mu}_2, \boldsymbol{\Sigma}_2)$, $\omega \in [0,1]$):

$$\boldsymbol{\Sigma}_f^{-1} = \omega\, \boldsymbol{\Sigma}_1^{-1} + (1-\omega)\, \boldsymbol{\Sigma}_2^{-1}$$

$$\boldsymbol{\Sigma}_f^{-1}\boldsymbol{\mu}_f = \omega\, \boldsymbol{\Sigma}_1^{-1}\boldsymbol{\mu}_1 + (1-\omega)\, \boldsymbol{\Sigma}_2^{-1}\boldsymbol{\mu}_2$$

Choose ω to minimise $\det(\boldsymbol{\Sigma}_f)$ or $\mathrm{tr}(\boldsymbol{\Sigma}_f)$.

Sequential algorithm (two inputs):

1. Precompute $\boldsymbol{\Sigma}_1^{-1}$, $\boldsymbol{\Sigma}_2^{-1}$.
2. **For each** ω in a grid over $[0,1]$:
   1. $\boldsymbol{\Lambda}_f(\omega) \leftarrow \omega\boldsymbol{\Sigma}_1^{-1} + (1-\omega)\boldsymbol{\Sigma}_2^{-1}$
   2. $\boldsymbol{\Sigma}_f(\omega) \leftarrow \boldsymbol{\Lambda}_f^{-1}$
   3. $\boldsymbol{\mu}_f(\omega) \leftarrow \boldsymbol{\Sigma}_f\left(\omega\boldsymbol{\Sigma}_1^{-1}\boldsymbol{\mu}_1 + (1-\omega)\boldsymbol{\Sigma}_2^{-1}\boldsymbol{\mu}_2\right)$
   4. Evaluate $J(\omega) = \log\det\boldsymbol{\Sigma}_f$ or $(\mathrm{tr}\,\boldsymbol{\Sigma}_f)$
3. Select $\omega^\star = \arg\min J(\omega)$; output $\boldsymbol{\mu}_f(\omega^\star)$, $\boldsymbol{\Sigma}_f(\omega^\star)$.

Key properties

- **1-D search**: efficient line search or closed-form in scalar case
- **Recomputed** at every fusion step (covariances change over time)
- **Fixed** $\omega = 0.5$: simplest option, more conservative
- Works for track-to-track fusion, multi-sensor networks, SLAM

Covariance Intersection (CI) — Interpretation

Geometric View: Convex Combination of Information Matrices

Define information matrices $\mathbf{\Lambda}_i = \mathbf{\Sigma}_i^{-1}$. CI forms

$$\mathbf{\Lambda}_f = \omega\,\mathbf{\Lambda}_1 + (1-\omega)\,\mathbf{\Lambda}_2 \quad \Longrightarrow \quad \mathbf{\Sigma}_f = \mathbf{\Lambda}_f^{-1}.$$

Because $\mathbf{\Lambda}_f$ is a *convex combination* of two PSD matrices, $\mathbf{\Sigma}_f$ is always a valid (conservative) upper bound on the true posterior — **regardless of** $\mathbf{\Sigma}_{12}$.

What does minimising the objective achieve?

Criterion	Meaning
$\min \det(\mathbf{\Sigma}_f)$	smallest *area* of uncertainty ellipse
$\min \mathrm{tr}(\mathbf{\Sigma}_f)$	smallest *average* variance

$\Rightarrow$ CI picks the **least conservative covariance that is still safe**:

tight, but never overconfident.

Summary of guarantees

- **Consistent**: $\mathbf{\Sigma}_f$ never underestimates true uncertainty
- **Conservative**: slightly pessimistic, but reliable
- **Robust**: no knowledge of $\mathbf{\Sigma}_{12}$ needed
- **Commutative & idempotent**: order of inputs does not matter; fusing an estimate with itself leaves it unchanged

Kalman Filter with Covariance Intersection (Two Sensors)

Goal: Fuse two sensor measurements when their correlation is unknown.

Step 1 — KF Prediction

$$\boldsymbol{\mu}_{t|t-1} = \boldsymbol{F}\boldsymbol{\mu}_{t-1|t-1}, \qquad \boldsymbol{\Sigma}_{t|t-1} = \boldsymbol{F}\boldsymbol{\Sigma}_{t-1|t-1}\boldsymbol{F}^\top + \boldsymbol{Q}$$

Step 2 — Local sensor updates
Update prediction separately using each sensor:
Sensor 1:

$$(\boldsymbol{\mu}_t^{(1)}, \boldsymbol{\Sigma}_t^{(1)})$$

Sensor 2:

$$(\boldsymbol{\mu}_t^{(2)}, \boldsymbol{\Sigma}_t^{(2)})$$

Step 3 — Fuse using Covariance Intersection

$$\Lambda_1 = (\boldsymbol{\Sigma}_t^{(1)})^{-1}, \qquad \Lambda_2 = (\boldsymbol{\Sigma}_t^{(2)})^{-1}$$

$$\Lambda_f(\omega) = \omega\Lambda_1 + (1-\omega)\Lambda_2$$

$$\boldsymbol{\Sigma}_{t|t} = \Lambda_f^{-1}, \qquad \boldsymbol{\mu}_{t|t} = \boldsymbol{\Sigma}_{t|t}(\omega\Lambda_1\boldsymbol{\mu}_t^{(1)} + (1-\omega)\Lambda_2\boldsymbol{\mu}_t^{(2)})$$

Choose $\omega \in [0,1]$ minimizing

$$\det\big(\boldsymbol{\Sigma}_{t|t}\big) \quad \text{or} \quad \mathrm{tr}(\boldsymbol{\Sigma}_{t|t})$$

CI vs Naïve KF — notebook ci_vs_kf_v2.ipynb

Why tr($\hat{P}$)/MSE is a Consistency Metric

For any **consistent** estimator, the covariance $\hat{\boldsymbol{P}}$ must satisfy $\underbrace{\mathrm{tr}(\hat{\boldsymbol{P}})}_{\text{claimed variance}} = \mathbb{E}\left[\|\hat{\mathbf{x}} - \mathbf{x}\|^2\right] = \underbrace{\mathrm{MSE}}_{\text{true average error}}$ because $\hat{\boldsymbol{P}} = \mathbb{E}[(\hat{\mathbf{x}} - \mathbf{x})(\hat{\mathbf{x}} - \mathbf{x})^\top]$ by definition. Empirically we estimate MSE via Monte Carlo, giving the ratio

$$\frac{\mathrm{tr}(\hat{\boldsymbol{P}})}{\mathrm{MSE}} \begin{cases} < 1 & \Rightarrow \text{underestimates uncertainty — } \textbf{overconfident} \\ \approx 1 & \Rightarrow \text{well-calibrated — optimal} \\ > 1 & \Rightarrow \text{overestimates uncertainty — conservative} \end{cases}$$

This is the scalar analogue of the **NEES** (Normalised Estimation Error Squared) test.

Filter	Mean tr(P)	Mean MSE	Ratio	Verdict
KF-Naïve	0.1206	0.1946	**0.619**	× Overconfident — MSE > tr(P)
KF-FullIR (oracle)	0.1958	0.1899	**1.031**	✓ Well-calibrated (optimal)
KF + CI	0.2036	0.1911	**1.066**	✓ Conservative (CI guarantee)

Key insight

All three filters track the target with *similar accuracy* (comparable MSE) — position estimates look nearly identical. The difference is entirely in **how honest each filter is about its own uncertainty**.

Practical implication:
In safety-critical systems (autonomous vehicles, aerospace) an *overconfident* covariance causes downstream modules (planners, controllers) to take decisions without adequate safety margins.

Learning Predictive Distributions per Modality

When the observation model is unknown or highly nonlinear, we learn an **approximate posterior** for each modality using neural networks.

Heteroscedastic Regression A neural network for modality m takes its measurement y_m (possibly high-dimensional, e.g. an image or radar scan) and outputs parameters of a Gaussian over the true state z:

$$\left(\mu^{(m)}(y_m),\ \sigma^{(m)2}(y_m)\right)$$

$$p^{(m)}(z \mid y_m) \ \approx\ \mathcal{N}\left(z;\ \mu^{(m)},\ \sigma^{(m)2}\right)$$

- $\sigma^{(m)2}$ captures **aleatoric uncertainty** (input-dependent noise)
- Learned by minimising the negative log-likelihood of z

Bayesian Approximations

- **MC Dropout**: stochastic forward passes
- **Deep Ensembles**: multiple independently trained models

These additionally capture **epistemic uncertainty** (model's own ignorance).

Output of each modality

Each sensor/modality produces an approximate posterior:

$$p^{(m)}(z \mid y_m) = \mathcal{N}\left(z;\ \mu^{(m)},\ \sigma^{(m)2}\right)$$

This is the learned analogue of the classical setup where $y_m = z + v_m$ with known σ_m^2: there too, each sensor yields a Gaussian belief about z.

Key idea Deep learning replaces the explicit observation model $p(y_m \mid z)$ with a learned approximate posterior

$$p^{(m)}(z \mid y_m)$$

The network ingests the raw measurement y_m and directly outputs a belief about z.

Fusion of Learned Posteriors

Each modality provides an approximate Gaussian posterior over z:

$$p^{(m)}(z \mid y_m) = \mathcal{N}\big(z;\ \mu^{(m)},\ \sigma^{(m)2}\big)$$

Under a flat (uninformative) prior on z, each posterior is proportional to the likelihood, so fusing posteriors reduces to multiplying likelihoods. Assuming conditional independence of measurements:

$$p(z \mid y_1, \ldots, y_M) \ \propto \ \prod_{m=1}^{M} p^{(m)}(z \mid y_m)$$

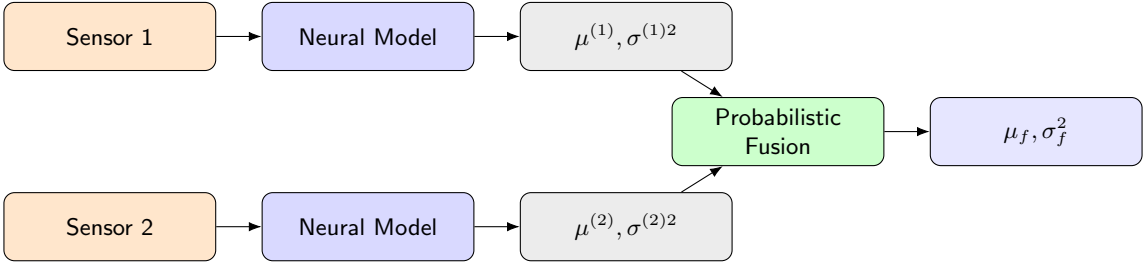
For Gaussian posteriors this yields the same **precision-weighted fusion** as the classical case:

$$\sigma_f^{-2} = \sum_{m=1}^{M} \sigma^{(m)-2} \qquad \hat{z} = \sigma_f^2 \sum_{m=1}^{M} \frac{\mu^{(m)}}{\sigma^{(m)2}}$$

Interpretation

Modalities with lower uncertainty (smaller $\sigma^{(m)2}$) receive larger fusion weight — exactly as sensor i with smaller σ_i^2 did in the classical N-sensor formula. The flat-prior assumption makes posterior fusion equivalent to precision-weighted averaging.

Multimodal Bayesian Fusion Pipeline



Deep models estimate uncertainty per modality; fusion combines the resulting distributions.

Product of Experts (PoE) Fusion

Assuming modality conditionals are independent given state $\boldsymbol{z}$:

$$p\left(\boldsymbol{z} \mid \boldsymbol{y}^{(1)}, \ldots, \boldsymbol{y}^{(M)}\right) \propto p(\boldsymbol{z}) \prod_{m=1}^{M} p\left(\boldsymbol{y}^{(m)} \mid \boldsymbol{z}\right)$$

For Gaussian experts $p_m = \mathcal{N}(\boldsymbol{z}; \boldsymbol{\mu}_m, \boldsymbol{\Sigma}_m)$:

$$\boldsymbol{\Sigma}_f^{-1} = \boldsymbol{\Sigma}_0^{-1} + \sum_m \boldsymbol{\Sigma}_m^{-1}$$

$$\boldsymbol{\mu}_f = \boldsymbol{\Sigma}_f \left(\boldsymbol{\Sigma}_0^{-1} \boldsymbol{\mu}_0 + \sum_m \boldsymbol{\Sigma}_m^{-1} \boldsymbol{\mu}_m \right)$$

Properties:

- Precision (confidence) additivity — same as information filter
- Overconfident if independence violated
- Uncertain expert (large $\boldsymbol{\Sigma}$) contributes small weight automatically
- Mixture-of-experts (MoE) is the dual: soft-selects one expert

Practical application

PoE fusion of IMU + GPS regression outputs. Degrade one modality at test time; watch uncertainty grow.

Block 3

Data Association

Relaxing the "known association" assumption

Previous Assumption: Single Target

In Blocks 1 and 2 we knew: y_1 cames from the target, y_2 cames from the same target.

Reality: at time k a sensor returns a *set* of measurements $\left\{y_k^{(1)}, \ldots, y_k^{(m)}\right\}$.

- Some originate from targets
- Some are **clutter** (false alarms)
- Each target detected with probability $P_D < 1$ (missed detections)
- The origin of each measurement is **unknown**

Data Association

Determine the mapping θ_k from measurements to tracks (or clutter) such that the resulting fusion is consistent.

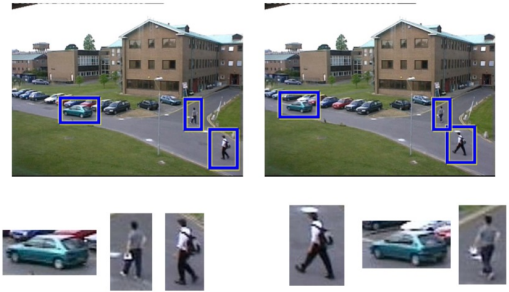
Data Association

The association problem is simply deciding which element in one set corresponds to which element in another set while minimizing cost. Tracking is one application, but the same mathematics is used in logistics, computer vision, robotics, and scheduling.

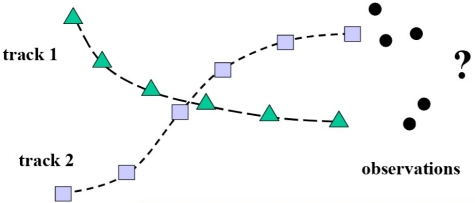
Data Association Ccenarios

Two-frame Matching (Correspondence Problem)

Match up detected blobs across frames



Multi-frame Matching (matching observations in a new frame to a set of tracked trajectories)



How to determine which observations to add to which track?

The figures are copied from "Introduction to Data Association," CSE598C Fall 2012 by Bob Collins.

Mahalanobis Distance

Suppose the predicted measurement distribution is

$$\mathbf{y} \sim \mathcal{N}(\hat{\mathbf{y}}, \mathbf{S})$$

where

$$\hat{\mathbf{y}} = \text{predicted measurement}, \qquad \mathbf{S} = \text{innovation covariance}.$$

The **Mahalanobis distance** between a measurement $\mathbf{y}$ and prediction $\hat{\mathbf{y}}$ is

$$D_M^2 = (\mathbf{y} - \hat{\mathbf{y}})^\top \mathbf{S}^{-1} (\mathbf{y} - \hat{\mathbf{y}})$$

Interpretation

- Euclidean distance measures

$$\|\mathbf{y} - \hat{\mathbf{y}}\|$$

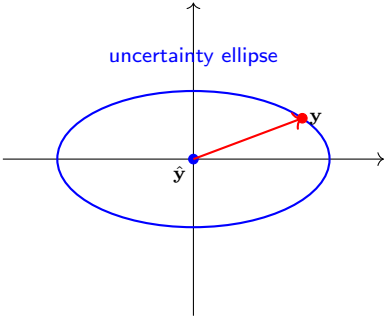
- Mahalanobis distance measures

distance in units of uncertainty

Example

- 1 m error with variance $0.01 \rightarrow$ very large distance
- 1 m error with variance $100 \rightarrow$ very small distance

Mahalanobis distance automatically scales errors by uncertainty.



Points with equal Mahalanobis distance form an **ellipse**.
The measurement error vector is simply $v = \mathbf{y} - \hat{\mathbf{y}}$

Gating is a simple filter for our sensor.

Imagine you are tracking an airplane on a radar.

- Your radar screen shows a blip moving steadily at 500 mph.
- Suddenly, the radar reports a new blip 50 miles away from the plane's last position, in just 1 second!

What do you do?
You immediately say: *"That's impossible. An airplane can't jump 50 miles in 1 second. That blip must be noise (a bird, a glitch, or clutter)."*

The Gating Concept

Gating is the mathematical version of this common sense. Before we do any complex tracking math, we ask the Kalman Filter: **"Where do you expect the object to be?"** We draw a "gate" (a boundary) around that expectation. Anything outside the gate is ignored.

Step 1: The Expectation and the Error (Innovation)

To build our gate, we first need to know where we expect the measurement to fall.

1. The Prediction ($\hat{\mathbf{y}}_t$)

The Kalman Filter predicts where the object is based on its past speed and heading. We convert this predicted state into a predicted measurement:

$$\hat{\mathbf{y}}_t = \mathbf{H}_t \, \boldsymbol{\mu}_{t|t-1}$$

Think of this as the center of our radar screen focus.

2. The Innovation ($\tilde{\mathbf{y}}_t$)

When a new measurement $\mathbf{y}_t$ actually arrives, we calculate the difference $\tilde{\mathbf{y}}_t = v_t$ between what we got and what we expected:

$$\tilde{\mathbf{y}}_t = \mathbf{y}_t - \hat{\mathbf{y}}_t$$

This is the "surprise factor." A small innovation means we were right. A huge innovation means something is wrong.

Step 2: How big should the gate be? (Covariance)

We can't just draw a fixed 5-mile circle around our prediction. What if the radar is really noisy today? What if the plane is maneuvering erratically?

The Innovation Covariance ($\mathbf{S}_t$)

The Kalman Filter calculates exactly how uncertain we are about our prediction. It combines two things:

1. **Model Uncertainty ($\mathbf{H}_t \, \mathbf{\Sigma}_{t|t-1} \, \mathbf{H}_t^\top$):** Are we unsure where the plane actually is?
2. **Sensor Noise ($\mathbf{R}_t$):** Is the radar blurry today?

$$\mathbf{S}_t = \underbrace{\mathbf{H}_t \, \mathbf{\Sigma}_{t|t-1} \, \mathbf{H}_t^\top}_{\text{Model Uncertainty}} + \underbrace{\mathbf{R}_t}_{\text{Sensor Noise}}$$

The Shape of the Gate

Because radars usually have different noise in distance vs. angle, our uncertainty is rarely a perfect circle. It forms an **ellipse** (an oval). $\mathbf{S}_t$ defines the shape and size of this ellipse.

Step 3: Drawing the Ellipsoidal Gate

Now we test our measurement. Is the "surprise" ($\tilde{\mathbf{y}}_t$) too big compared to our uncertainty ($\mathbf{S}_t$)?
The Mahalanobis Distance (D^2): We measure distance using our uncertainty ellipse:

$$D^2 = \tilde{\mathbf{y}}_t^\top \mathbf{S}_t^{-1} \tilde{\mathbf{y}}_t$$

Mahalanobis distance measures how many standard deviations the measurement is from the prediction. If the prediction error is Gaussian, the squared Mahalanobis distance follows a Chi-Square distribution $D_M^2 \sim \chi_k^2$, where k is the measurement dimension. Therefore we can choose a threshold that corresponds to a desired confidence region.

The Threshold (γ): We set a maximum allowed distance γ. If $D^2 \leq \gamma$, the measurement passes the gate!

$$\tilde{\mathbf{y}}_t^\top \mathbf{S}_t^{-1} \tilde{\mathbf{y}}_t \leq \gamma$$

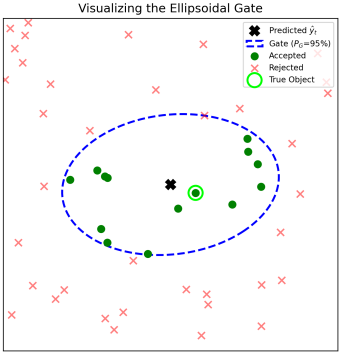
Step 4: Choosing γ (The Confidence Level)

How do we pick γ? The gate threshold γ is chosen from **Chi-Squared (χ^2)** distribution so that $P(D_M^2 < \gamma) = \alpha$.

1. We decide how confident we want to be (P_G). "I want my gate to catch the true airplane 99% of the time." ($P_G = 0.99$).
2. We look up 0.99 in a Chi-Squared table (based on how many dimensions our radar has, e.g., $k = 2$).
3. The table tells us $\gamma = 9.21$.

Gate size trade-off

Too small ($P_G = 0.50$): We might accidentally reject the real airplane!
Too large ($P_G = 0.999$): The gate is huge. We let in way too much noise (clutter).



Gating: Worked Numeric Example

2-D gating: which measurements pass?

Setup: Predicted measurement $\hat{\boldsymbol{y}} = (10, 5)$, innovation covariance $\boldsymbol{S} = \begin{pmatrix} 4.25 & 3.0 \\ 3.0 & 5.25 \end{pmatrix}$, $\boldsymbol{S}^{-1} = \begin{pmatrix} 0.447 & -0.256 \\ -0.256 & 0.362 \end{pmatrix}$.

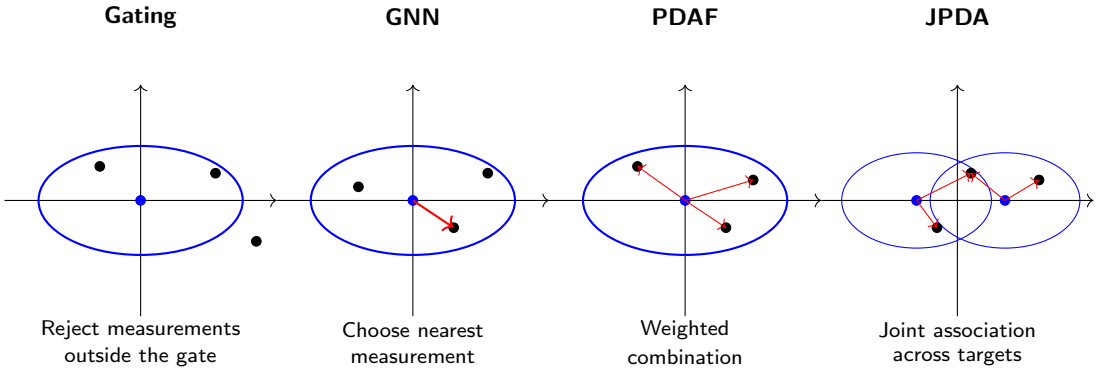
Gate threshold: $P_G = 0.99$, $k = 2 \Rightarrow \gamma = \chi^2_2(0.99) = 9.21$.

Test 4 candidate measurements:

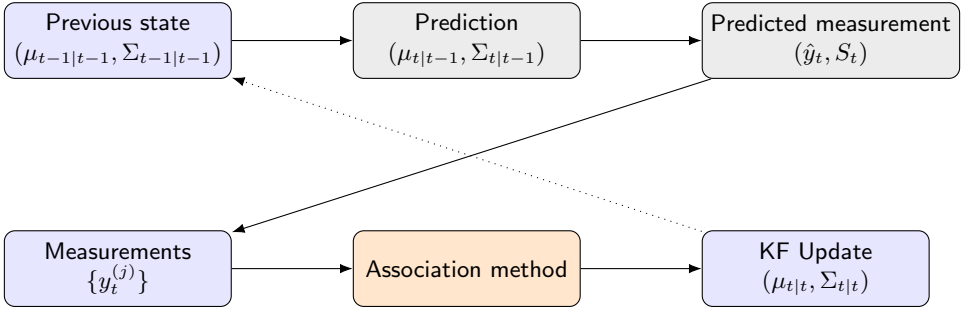
Point	$\tilde{\boldsymbol{y}} = \boldsymbol{y} - \hat{\boldsymbol{y}}$	$d_M^2 = \tilde{\boldsymbol{y}}^\top \boldsymbol{S}^{-1} \tilde{\boldsymbol{y}}$	$d_M^2 \leq \gamma$?	Decision
$P_1 = (13, 5)$	$(3, 0)$	4.02	$4.02 \leq 9.21$ ✓	Inside
$P_2 = (10, 8)$	$(0, 3)$	3.26	$3.26 \leq 9.21$ ✓	Inside
$P_3 = (12, 7)$	$(2, 2)$	2.27	$2.27 \leq 9.21$ ✓	Inside
$P_4 = (15, 1)$	$(5, -4)$	13.57	$13.57 > 9.21$ ×	Outside

P_4 has Euclidean distance $\|\tilde{\boldsymbol{y}}\| = 6.4$, similar to P_1's 3.0 + P_2's 3.0, yet is *rejected* because it lies along a high-precision (low-variance) direction.

Data Association Methods: Comparison



Where Data Association Enters the Kalman Filter



Association methods used inside the orange block:

- **Gating:** reject unlikely measurements
- **GNN:** choose the closest measurement
- **PDAF:** probabilistic combination of measurements
- **JPDA:** joint assignment across multiple tracks

Global Nearest Neighbour (GNN): Hard Assignment

Innovation (measurement residual)

$$v_{ij} = y_j - \hat{y}_i$$

- y_j = measurement j
- $\hat{y}_i$ = predicted measurement of target i

Mahalanobis distance

$$D_{ij}^2 = v_{ij}^\top S_i^{-1} v_{ij}$$

Cost matrix

$$C_{ij} = D_{ij}^2$$

- small cost $\rightarrow$ good match
- large cost $\rightarrow$ poor match
- If outside the gate $\rightarrow C_{ij} = \infty$

GNN assignment problem

$$\min_{\Theta \in \{0,1\}^{T \times M}} \sum_{i,j} C_{ij} \Theta_{ij}$$

subject to $\sum_j \Theta_{ij} \leq 1$ and $\sum_i \Theta_{ij} \leq 1$

- each target gets at most one measurement
- each measurement used at most once

Solved using the **Hungarian algorithm**.

- solves the **linear assignment problem** (LAP): given a cost matrix $\mathbf{C} \in \mathbb{R}^{n \times n}$, find a one-to-one assignment (a permutation) that minimizes the total cost $\sum_i C_{i,\pi(i)}$.
- runs in $O(n^3)$ time
- matching problems (e.g., matching detections between frames)

Example: Assignment Using the Cost Matrix

Step 1 — Compute matching cost

$$C_{ij} = D_{ij}^2$$

where

$$D_{ij}^2 = v_{ij}^\top S_i^{-1} v_{ij}$$

Small cost means the measurement fits the predicted target well.

Step 2 — Build the cost table

	Measurement 1	Measurement 2	Measurement 3
Target 1	1.2	5.3	∞
Target 2	4.1	0.7	3.0
Target 3	∞	2.1	0.5

Each entry represents the matching cost.

Step 3 — Hungarian algorithm

The Hungarian algorithm automatically finds this optimal assignment.

Step 3.1 — Resulting assignment

Target	Measurement
Target 1	Measurement 1
Target 2	Measurement 2
Target 3	Measurement 3

Step 3.2 — Minimal total cost

$$1.2 + 0.7 + 0.5 = 2.4$$

Global Nearest Neighbour (GNN): Problems

Problems:

- Hard commitment — wrong assignment propagated as *certain*
- Track covariance does not grow despite association uncertainty
- Two crossing targets: GNN swaps identities at the crossing
- No self-healing: a wrong assignment at step k poisons step $k+1$

Why hard assignment is dangerous

Treating an uncertain association as certain is equivalent to *ignoring aleatoric uncertainty about measurement origin*. The posterior covariance is underestimated.

Probabilistic Data Association Filter (PDAF): Association Events — Worked Example

PDAF increases covariance when candidate measurements disagree, because the filter is uncertain about which measurement generated the observation.
Setup: At time t, after gating we have $m = 3$ observations y_1, y_2, y_3.

Mutually exclusive

- θ_j $(j = 1, 2, 3)$: observation j *originated from the target*
- θ_0: *none* of the gated observations came from the target

The association probabilities are $\beta_j = P(\theta_j | \boldsymbol{y})$. These are mutually exclusive and exhaustive: $\sum_{j=0}^{3} \beta_j = 1$.

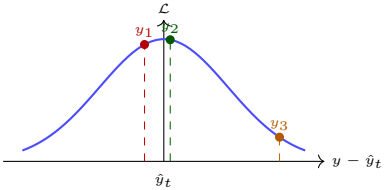
Likelihood of each observation:

How consistent is y_j with what the filter *predicts* it should see?

$$\mathcal{L}_j = \mathcal{N}(y_j;\ \hat{y}_t,\ S_t)\,, \quad \hat{y}_t = \mathbf{H}\,\hat{\mathbf{y}}_{t|t-1}$$

Numeric example (1-D, $S_t = 1$):

Obs.	$y_j - \hat{y}_t$	$\mathcal{L}_j$
y_1	-0.3	0.381
y_2	$+0.1$	0.397
y_3	$+1.8$	0.079



y_3 is far from $\hat{y}_t$ $\Rightarrow$ low likelihood $\Rightarrow$ low weight.

PDAF: Computing Association Probabilities β_j

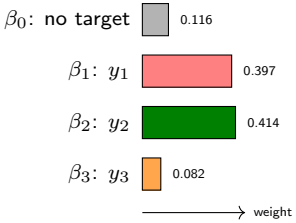
$$\beta_j = \frac{P_D\,\mathcal{L}_j}{\lambda\bar{c} + P_D \sum_{j'=1}^{m} \mathcal{L}_{j'}}, \qquad \beta_0 = \frac{\lambda\bar{c}}{\lambda\bar{c} + P_D \sum_{j'=1}^{m} \mathcal{L}_{j'}}$$

where P_D = detection probability, λ = clutter density, $\bar{c} = 1 - P_D P_G$.

Numeric example: $P_D = 0.9,\ \ \lambda\bar{c} = 0.10$

$$\text{denom} = 0.10 + 0.9 \times (0.381 + 0.397 + 0.079) = 0.864$$

Event	Formula	β
θ_0	$0.10/0.864$	**0.116**
θ_1	$0.9 \times 0.381/0.864$	**0.397**
θ_2	$0.9 \times 0.397/0.864$	**0.414**
θ_3	$0.9 \times 0.079/0.864$	**0.082**
Sum		1.000



y_1 and y_2 carry most of the weight.
y_3 gets only 8% — it is far from $\hat{y}_t$.
11% probability: target not detected at all.

PDAF: Why the Association Probabilities β_j Look This Way

Goal: Compute the probability that measurement y_j originated from the target among the m measurements in the validation gate.

$$\beta_j = P(\theta_j \mid Y), \qquad Y = \{y_1, \ldots, y_m\}.$$

Using Bayes' rule:

$$P(\theta_j \mid Y) = \frac{P(Y \mid \theta_j)\, P(\theta_j)}{\sum_{k=0}^{m} P(Y \mid \theta_k)\, P(\theta_k)}.$$

Model assumptions (PDAF):

- If y_j is the target-originated measurement: $P(Y \mid \theta_j) = \mathcal{L}_j$ (KF innovation likelihood for measurement y_j).
- If none of the measurements come from the target: $P(Y \mid \theta_0) = \lambda\, \bar{c}$, where λ is the clutter spatial density and $\bar{c} = 1 - P_D P_G$.
- Priors: $P(\theta_j) = P_D P_G$ and $P(\theta_0) = \bar{c}$.

This yields the PDAF weights:

$$\beta_j = \frac{P_D\, \mathcal{L}_j}{\lambda \bar{c} + P_D \sum_{i=1}^{m} \mathcal{L}_i}, \qquad \beta_0 = \frac{\lambda \bar{c}}{\lambda \bar{c} + P_D \sum_{i=1}^{m} \mathcal{L}_i}.$$

Where do the parameters come from?

- P_D: detection probability — from sensor specs; sometimes adaptively estimated.
- P_G: gate probability — computed from KF innovation covariance and χ^2 gating: $P_G = F_{\chi^2_k}(\gamma)$.
- λ: clutter spatial density — estimated from environment statistics or assumed constant.

PDAF Mean Update — Step-by-Step Derivation

Goal: find $\hat{\boldsymbol{\mu}}_{t|t} = \mathbb{E}[\boldsymbol{\mu}_t \mid \mathcal{Y}_t]$.

Step 1 — Condition on each association event (law of total expectation):

$$\hat{\boldsymbol{\mu}}_{t|t} = \sum_{j=0}^{m} \beta_j \underbrace{\mathbb{E}[\boldsymbol{\mu}_t \mid \theta_j,\, \mathcal{Y}_t]}_{\text{what would the update be IF } \theta_j \text{ were true?}}$$

Step 2 — Evaluate each conditional update:

- If θ_0 (no valid obs): no update possible $\Rightarrow$ $\mathbb{E}[\boldsymbol{\mu}_t \mid \theta_0] = \hat{\boldsymbol{\mu}}_{t|t-1}$
- If θ_j (y_j is true): standard KF update with that one obs $\Rightarrow$ $\mathbb{E}[\boldsymbol{\mu}_t \mid \theta_j] = \hat{\boldsymbol{\mu}}_{t|t-1} + \mathbf{K}_t\, \mathbf{v}_j$

where $\mathbf{v}_j = y_j - \hat{y}_t$ is the **innovation** for observation j.

Step 3 — Substitute back:

$$\hat{\boldsymbol{\mu}}_{t|t} = \beta_0\, \hat{\boldsymbol{\mu}}_{t|t-1} + \sum_{j=1}^{m} \beta_j \left(\hat{\boldsymbol{\mu}}_{t|t-1} + \mathbf{K}_t \mathbf{v}_j\right) = \hat{\boldsymbol{\mu}}_{t|t-1} + \mathbf{K}_t \underbrace{\sum_{j=1}^{m} \beta_j\, \mathbf{v}_j}_{\bar{\mathbf{v}}}$$

Result

$$\hat{\boldsymbol{\mu}}_{t|t} = \hat{\boldsymbol{\mu}}_{t|t-1} + \mathbf{K}_t\, \bar{\mathbf{v}}, \qquad \bar{\mathbf{v}} = \sum_{j=1}^{m} \beta_j\, \mathbf{v}_j$$

$\bar{\mathbf{v}}$ is the **combined innovation**: a probability-weighted average of all individual innovations. Note β_0 drops out because its contribution to the innovation sum is zero.

PDAF Mean Update — Numeric Example & Geometric View

From the previous slide: $\hat{y}_t = 0,\ K = 0.5$

$$v_1 = -0.3, \quad v_2 = +0.1, \quad v_3 = +1.8$$

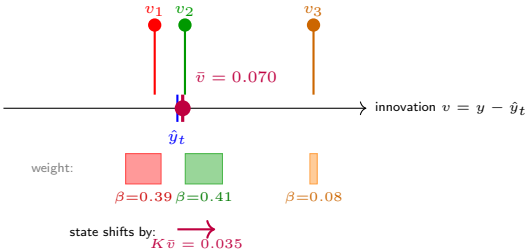
Combined innovation:

$$\bar{v} = 0.397(-0.3) + 0.414(0.1) + 0.082(1.8)$$

$$\bar{v} = -0.119 + 0.041 + 0.148 = +0.070$$

State update:

$$\hat{y}_{t|t} = \hat{y}_{t|t-1} + K\bar{v} = \hat{y}_{t|t-1} + 0.035$$



Interpretation:
Instead of jumping all the way to a single observation, the state moves by only $K\bar{v} = 0.035$ — a small step pulled roughly toward where y_1 and y_2 cluster, with y_3 contributing little.

The bars below each observation show its weight β_j. y_1 and y_2 have large bars and pull $\bar{v}$ toward them. y_3 has a tiny bar — it barely moves the state.

PDAF Covariance Update — The Spread Term

$$\boxed{\mathbf{\Sigma}_{t|t} = \underbrace{\beta_0\,\mathbf{\Sigma}_{t|t-1}}_{\text{(A) no detection}} + \underbrace{(1-\beta_0)\,\mathbf{\Sigma}^*_{t|t}}_{\text{(B) standard KF update}} + \underbrace{\mathbf{K}_t\left[\sum_{j=1}^{m}\beta_j\mathbf{v}_j\mathbf{v}_j^\top - \bar{\mathbf{v}}\bar{\mathbf{v}}^\top\right]\mathbf{K}_t^\top}_{\tilde{\mathbf{\Sigma}}\,\geq\,0\ \text{(C) spread of innovations}}}$$

where $\mathbf{\Sigma}^*_{t|t} = (\mathbf{I} - \mathbf{K}_t\mathbf{H}_t)\mathbf{\Sigma}_{t|t-1}$.

Three terms explained:

(A) With prob. β_0: no measurement updated us — covariance stays at predicted level.

(B) With prob. $1-\beta_0$: a true target return was received — covariance shrinks like a standard KF update.

(C) $\tilde{\mathbf{\Sigma}}$ is the **weighted variance** of innovations around their mean $\bar{\mathbf{v}}$. It captures "how spread out are the candidate observations?" Always ≥ 0, so uncertainty never artificially shrinks.

Numeric example (1-D, $K = 0.5$):

$$\sum_j \beta_j v_j^2 = 0.397(0.09) + 0.414(0.01) + 0.082(3.24) = 0.306$$

$$\bar{v}^2 = (0.070)^2 = 0.005$$

$$\tilde{\Sigma} = K^2(0.306 - 0.005) = 0.25 \times 0.301 = 0.075$$

Effect of clutter

Heavy clutter $\Rightarrow$ β_j nearly equal $\Rightarrow$ innovations spread widely $\Rightarrow$ $\tilde{\Sigma}$ large $\Rightarrow$ $\mathbf{\Sigma}_{t|t}$ inflates.
The filter becomes more cautious automatically.

Examples of PDAF Measurement–to–Track Association

We track two objects with IDs:

$$\text{Track 1: ID1}, \qquad \text{Track 2: ID2}$$

with predicted measurements:

$$\hat{y}_1 = 10, \qquad \hat{y}_2 = 30.$$

A) One measurement arrives: $Y = \{\, y_1 = 12 \,\}$

- Gate(ID1) contains 12 (close to 10)
- Gate(ID2) does *not* contain 12 (too far from 30)
- PDAF:

$$\beta_1^{(ID1)} \approx 1, \qquad \beta_0^{(ID2)} \approx 1.$$

Interpretation: Measurement 12 belongs to Track ID1 with $\approx 100\%$ probability. Track ID2 experiences a missed detection.

B) Three measurements arrive: $Y = \{\, 11,\ 29,\ 40 \,\}$

Track ID1 (pred. 10)

- Gate contains only $y_1 = 11$
- PDAF:

$$\beta_1^{(ID1)} \approx 1, \qquad \beta_0^{(ID1)} \approx 0.$$

Track ID2 (pred. 30)

- Gate contains $y_2 = 29$ and $y_3 = 40$
- Likelihoods: $\mathcal{L}_2 \gg \mathcal{L}_3$
- PDAF:

$$\beta_2^{(ID2)} \approx 0.85, \qquad \beta_3^{(ID2)} \approx 0.10, \qquad \beta_0^{(ID2)} \approx 0.05.$$

Interpretation:

- Measurement 11 is assigned to Track ID1 with $\approx 100\%$ probability.
- For Track ID2, measurement 29 has the highest association probability ($\approx 85\%$), but measurement 40 still contributes weakly ($\approx 10\%$).
- No "hard" assignment: PDAF uses all gated measurements according to β_j.

PDAF Self-Consistency: Three Limiting Cases

We verify the formula behaves correctly at the extremes.

Case 1: one observation

Situation: exactly one gated observation, almost certain it is the target.

$\bar{v} = \beta_1 v_1 = v_1$ (only one term)
$\tilde{\Sigma} = K^2[\beta_1 v_1^2 - \bar{v}^2] = K^2[v_1^2 - v_1^2] = 0$
$\mathbf{\Sigma}_{t|t} = \mathbf{\Sigma}^*_{t|t}$ ✓

Conclusion: PDAF reduces exactly to the standard KF. The spread term vanishes because there is no ambiguity about which observation to use.

Case 2: $\beta_0 \to 1$

Situation: no gated observations (or all are clutter). Target not detected.

$\bar{v} = \sum_j \beta_j v_j = 0$ (no $j \geq 1$ terms)
$\tilde{\Sigma} = 0$
$\mathbf{\Sigma}_{t|t} = \beta_0 \mathbf{\Sigma}_{t|t-1} = \mathbf{\Sigma}_{t|t-1}$ ✓

Conclusion: State does not move; covariance does not shrink. Correct — receiving no information should not make the filter more confident.

Case 3: dense clutter

Situation: m large, weights flat: $\beta_j \approx \frac{1}{m+1}$.

$\bar{v} \approx 0$ (contributions cancel)
$\tilde{\Sigma} \approx K^2 \cdot \frac{1}{m} \sum_j v_j^2$ (large)
$\mathbf{\Sigma}_{t|t} \approx \mathbf{\Sigma}^*_{t|t} + \tilde{\Sigma} > \mathbf{\Sigma}^*_{t|t}$ ✓

Conclusion: Covariance inflates with clutter density. The filter widens its effective uncertainty automatically — no manual threshold needed.

Summary: PDAF is self-regulating

Condition	Innovation $\bar{v}$	Covariance effect
1 obs., $\beta_0 \to 0$	$= v_1$	shrinks like standard KF
No detection, $\beta_0 \to 1$	$= 0$	unchanged
Dense clutter	≈ 0	inflates (spread term large)

PDAF — Complete Algorithm (per time step)

1. **Predict** (standard KF):

$$\hat{\boldsymbol{\mu}}_{t|t-1} = \mathbf{F}\hat{\boldsymbol{\mu}}_{t-1|t-1}, \quad \boldsymbol{\Sigma}_{t|t-1} = \mathbf{F}\boldsymbol{\Sigma}_{t-1|t-1}\mathbf{F}^\top + \mathbf{Q}$$

2. **Gate**: keep observations y_j satisfying

$$(y_j - \hat{y}_t)^\top \mathbf{S}_t^{-1}(y_j - \hat{y}_t) \;\le\; \gamma$$

3. **Compute likelihoods**:

$$\mathcal{L}_j = \mathcal{N}(y_j;\, \hat{y}_t,\, \mathbf{S}_t)$$

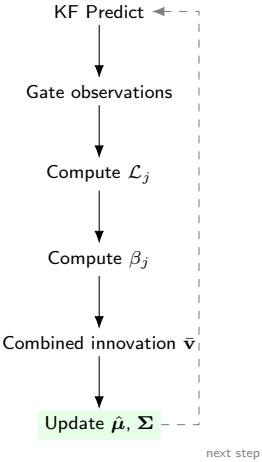
4. **Compute association weights** β_j, β_0.
5. **Innovations and combined innovation**:

$$\mathbf{v}_j = y_j - \hat{\mathbf{y}}_t, \qquad \bar{\mathbf{v}} = \sum_{j=1}^{m} \beta_j\, \mathbf{v}_j$$

6. **Update mean**:

$$\hat{\boldsymbol{\mu}}_{t|t} = \hat{\boldsymbol{\mu}}_{t|t-1} + \mathbf{K}_t\bar{\mathbf{v}}$$

7. **Update covariance** (include spread term $\tilde{\boldsymbol{\Sigma}}$).



Track Coalescence — Step-by-Step Mechanism

How do two tracks physically merge? Follow the update equations for both tracks over a few steps.

One time step in detail:

Both tracks share the same gated observation y_j. Their combined innovations are:

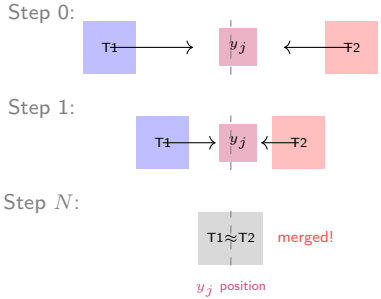
$$\bar{v}_1 = \beta_{1j}(y_j - \hat{y}_1) = 0.8\,(y_j - \hat{y}_1)$$

$$\bar{v}_2 = \beta_{2j}(y_j - \hat{y}_2) = 0.8\,(y_j - \hat{y}_2)$$

Both state estimates shift **strongly toward** y_j.

After the update:

- $\hat{y}_1$ moves *right* (toward y_j)
- $\hat{y}_2$ moves *left* (toward y_j)
- After several steps: $\hat{y}_1 \approx \hat{y}_2 \approx y_j$



Why recovery is hard: Once the two covariance ellipses overlap completely, the filter cannot distinguish which measurements belong to which target. Even if the targets separate physically, both tracks may continue following the same cluster of measurements.

Track Coalescence — Consequences & Fix (JPDA)

When does coalescence happen?

- Targets come **close together** (crossing, formation flight, convoy tracking)
- Gates **overlap** so both tracks gate the same observations
- **High clutter**: many spurious returns make the weights β_{ij} even more diffuse

Real-world impact:

- **Air traffic control:** two aircraft appear as one
- **Autonomous vehicles:** two pedestrians merge into one
- **Multi-robot:** teammates appear at the same location

Root cause

PDAF's association probabilities are **marginal** per-track probabilities. They do not enforce the physical constraint that *one observation can originate from at most one target*.

The fix: Joint Probabilistic Data Association (JPDA)

JPDA replaces per-track marginal probabilities with **joint** association events Θ over all tracks simultaneously.

How JPDA Works — The Big Picture

1. **Gate** — For each track, find which observations fall inside its validation gate (same as PDAF).
2. **Enumerate** — List every *legal* way to assign observations to tracks. "Legal" means no observation is shared by two tracks.
3. **Weight** — Give each legal assignment a probability based on how well each observation fits each track (the likelihoods $\mathcal{L}_j$).
4. **Marginalise** — For each track, sum the probabilities of all assignments where it gets a particular observation. These sums are the β weights.

After step 4, each track is updated using the **standard PDAF / Kalman Filter** update equations — only the β weights are different.

Enumerating Valid Joint Events

Scenario: 2 tracks (T_1, T_2) and 2 observations (y_1, y_2). Both observations gate to both tracks.

Event	Description	$T_1 \leftarrow$	$T_2 \leftarrow$	Valid?	Why?
Θ_1	Straight	y_1	y_2	✓	Each obs. used once
Θ_2	Swapped	y_2	y_1	✓	Still valid
Θ_3	T_2 missed	y_1	—	✓	T_2 undetected $(1-P_D)$
Θ_4	T_2 missed	y_2	—	✓	T_2 undetected $(1-P_D)$
Θ_5	T_1 missed	—	y_1	✓	T_1 undetected $(1-P_D)$
Θ_6	T_1 missed	—	y_2	✓	T_1 undetected $(1-P_D)$
—	Illegal	y_1	y_1	×	y_1 used twice!

The last row violates the constraint: one observation cannot originate from two targets simultaneously.

JPDA Algorithm — Step by Step

Inputs: $\hat{\boldsymbol{\mu}}_{i,\,t|t-1}$, $\boldsymbol{\Sigma}_{i,\,t|t-1}$ for each track $i = 1 \ldots T$ (from KF predict); observations $\{y_1, \ldots, y_m\}$.

1. **Gate.** Compute innovation $\mathbf{v}_{ij} = y_j - \hat{y}_{i,t}$. Keep y_j for track i if $\mathbf{v}_{ij}^\top \boldsymbol{S}_{i,t}^{-1} \mathbf{v}_{ij} \leq \gamma$ (chi-squared threshold).
2. **Enumerate.** List all valid joint events Θ where each observation is assigned to *at most one* target.
3. **Weight.** For each event Θ:
   $P(\Theta) \propto \prod_{\text{det. pairs } (i,j)} P_D \cdot \mathcal{N}\big(\mathbf{v}_{ij};\, \mathbf{0},\, \boldsymbol{S}_{i,t}\big) \times \prod_{\text{missed}} (1 - P_D) \times \lambda^{\#\text{false alarms}}$. Normalise so $\sum_\Theta P(\Theta) = 1$.
4. **Marginalise.** $\beta_{ij} = \sum_{\Theta:\ \text{target } i \text{ gets obs } j} P(\Theta)$, $\beta_{i0} = \sum_{\Theta:\ \text{target } i \text{ undetected}} P(\Theta)$.
5. **Update.** Feed β_{ij} into the PDAF update equations (next slide).

Probability $P(\Theta)$ is obtained by multiplying:

- How likely the detection is (Gaussian)
- How likely the sensor detects a target (P_D)
- How likely missed detections occur ($1 - P_D$)
- How likely false alarms occur (λ)

Merging JPDA with the Kalman Filter

Standard KF Predict (unchanged)

$$\hat{\boldsymbol{\mu}}_{t|t-1} = \boldsymbol{F}\,\hat{\boldsymbol{\mu}}_{t-1|t-1}$$

$$\boldsymbol{\Sigma}_{t|t-1} = \boldsymbol{F}\,\boldsymbol{\Sigma}_{t-1|t-1}\,\boldsymbol{F}^{\top} + \boldsymbol{Q}$$

JPDA provides the β weights

$$\beta_{ij} = P(\text{obs } j \to \text{track } i \mid \text{all data})$$

$$\beta_{i0} = P(\text{track } i \text{ has no obs.})$$

Innovations: $\mathbf{v}_{ij} = y_j - \hat{\mathbf{y}}_{i,t}, \qquad \bar{\mathbf{v}}_i = \sum_{j=1}^{m} \beta_{ij}\,\mathbf{v}_{ij}$ (combined innov.)

Update mean: $\hat{\boldsymbol{\mu}}_{i,\,t|t} = \hat{\boldsymbol{\mu}}_{i,\,t|t-1} + \boldsymbol{K}_{i,t}\,\bar{\mathbf{v}}_i$

Update cov.: $\boldsymbol{\Sigma}_{i,\,t|t} = \beta_{i0}\,\boldsymbol{\Sigma}_{i,\,t|t-1} + (1-\beta_{i0})\,\boldsymbol{\Sigma}_i^{\mathsf{KF}} + \underbrace{\boldsymbol{K}_{i,t}\Big[\sum_j \beta_{ij}\,\mathbf{v}_{ij}\mathbf{v}_{ij}^{\top} - \bar{\mathbf{v}}_i\bar{\mathbf{v}}_i^{\top}\Big]\boldsymbol{K}_{i,t}^{\top}}_{\tilde{\boldsymbol{\Sigma}}_i \text{ (spread term)}}$

Worked Example: Two Planes, Two Blips

[Setup] $P_D = 0.9$, clutter density $\lambda = 0.1$. Both observations gate to both tracks.

Likelihoods (how well each observation fits each track):

$\mathcal{L}_{11} = \mathcal{N}(y_1;\, \hat{y}_1,\, \boldsymbol{S}_1) = 0.50$ $\quad \mathcal{L}_{12} = \mathcal{N}(y_2;\, \hat{y}_1,\, \boldsymbol{S}_1) = 0.05$ $\quad$ (T_1: y_1 fits well, y_2 poorly)

$\mathcal{L}_{21} = \mathcal{N}(y_1;\, \hat{y}_2,\, \boldsymbol{S}_2) = 0.10$ $\quad \mathcal{L}_{22} = \mathcal{N}(y_2;\, \hat{y}_2,\, \boldsymbol{S}_2) = 0.40$ $\quad$ (T_2: y_2 fits well, y_1 poorly)

Event	$T_1 \leftarrow$	$T_2 \leftarrow$	Weight formula	Calculation	Value
Θ_1	y_1	y_2	$P_D^2\, \mathcal{L}_{11}\mathcal{L}_{22}$	$0.81 \times 0.50 \times 0.40$	0.1620
Θ_2	y_2	y_1	$P_D^2\, \mathcal{L}_{12}\mathcal{L}_{21}$	$0.81 \times 0.05 \times 0.10$	0.0041
Θ_3	y_1	—	$P_D(1-P_D)\, \mathcal{L}_{11}/\lambda$	0.09×0.50 / 0.1	0.4500
Θ_4	y_2	—	$P_D(1-P_D)\, \mathcal{L}_{12}/\lambda$	0.09×0.05 / 0.1	0.0450
Θ_5	—	y_1	$(1-P_D)P_D\, \mathcal{L}_{21}/\lambda$	0.09×0.10 / 0.1	0.0900
Θ_6	—	y_2	$(1-P_D)P_D\, \mathcal{L}_{22}/\lambda$	0.09×0.40 / 0.1	0.3600

Total $= 0.162 + 0.004 + 0.450 + 0.045 + 0.090 + 0.360 = 1.111$. Normalise: divide each value by 1.111 to get $P(\Theta)$.

Example Results: The β Weights

Normalised event probabilities: $P(\Theta_1)$=0.146, $P(\Theta_2)$=0.004, $P(\Theta_3)$=0.405, $P(\Theta_4)$=0.041, $P(\Theta_5)$=0.081, $P(\Theta_6)$=0.324.

Track 1 (T_1)

$$\beta_{1,1} = P(\Theta_1)+P(\Theta_3) = 0.146+0.405 = \mathbf{0.551}$$
$$\beta_{1,2} = P(\Theta_2)+P(\Theta_4) = 0.004+0.041 = \mathbf{0.044}$$
$$\beta_{1,0} = P(\Theta_5)+P(\Theta_6) = 0.081+0.324 = \mathbf{0.405}$$

Check: $0.551 + 0.044 + 0.405 = 1.000$ ✓

Track 2 (T_2)

$$\beta_{2,1} = P(\Theta_2)+P(\Theta_5) = 0.004+0.081 = \mathbf{0.085}$$
$$\beta_{2,2} = P(\Theta_1)+P(\Theta_6) = 0.146+0.324 = \mathbf{0.470}$$
$$\beta_{2,0} = P(\Theta_3)+P(\Theta_4) = 0.405+0.041 = \mathbf{0.446}$$

Check: $0.085 + 0.470 + 0.446 = 1.000$ ✓

Marginalising for Track 1. To compute each β weight for T_1, scan the "$T_1 \leftarrow$" column and group the events by which observation T_1 receives:

- $\beta_{1,1}$ (Track 1 gets y_1): The "$T_1 \leftarrow$" column contains y_1 in events Θ_1 and Θ_3. It does not matter what T_2 does in those events — we only care that T_1 was assigned y_1.
- $\beta_{1,2}$ (Track 1 gets y_2): T_1 receives y_2 in events Θ_2 and Θ_4.
- $\beta_{1,0}$ (Track 1 gets nothing): T_1 is undetected ("—") in events Θ_5 and Θ_6.

Worked Example: Two Planes, Two Blips

[Setup] $P_D = 0.9$, clutter density $\lambda = 0.1$. Both observations gate to both tracks.

Likelihoods (how well each observation fits each track):

$\mathcal{L}_{11} = \mathcal{N}(y_1;\, \hat{y}_1,\, \boldsymbol{S}_1) = 0.50$ $\quad \mathcal{L}_{12} = \mathcal{N}(y_2;\, \hat{y}_1,\, \boldsymbol{S}_1) = 0.05$ $\quad$ (T_1: y_1 fits well, y_2 poorly)

$\mathcal{L}_{21} = \mathcal{N}(y_1;\, \hat{y}_2,\, \boldsymbol{S}_2) = 0.10$ $\quad \mathcal{L}_{22} = \mathcal{N}(y_2;\, \hat{y}_2,\, \boldsymbol{S}_2) = 0.40$ $\quad$ (T_2: y_2 fits well, y_1 poorly)

Event	$T_1 \leftarrow$	$T_2 \leftarrow$	Weight formula	Calculation	Value
Θ_1	y_1	y_2	$P_D^2\, \mathcal{L}_{11}\mathcal{L}_{22}\, \lambda^0$	$0.81 \times 0.50 \times 0.40$	0.16200
Θ_2	y_2	y_1	$P_D^2\, \mathcal{L}_{12}\mathcal{L}_{21}\, \lambda^0$	$0.81 \times 0.05 \times 0.10$	0.00405
Θ_3	y_1	—	$P_D(1-P_D)\, \mathcal{L}_{11}\, \lambda$	$0.9 \times 0.1 \times 0.50 \times 0.1$	0.00450
Θ_4	y_2	—	$P_D(1-P_D)\, \mathcal{L}_{12}\, \lambda$	$0.9 \times 0.1 \times 0.05 \times 0.1$	0.00045
Θ_5	—	y_1	$(1-P_D)P_D\, \mathcal{L}_{21}\, \lambda$	$0.1 \times 0.9 \times 0.10 \times 0.1$	0.00090
Θ_6	—	y_2	$(1-P_D)P_D\, \mathcal{L}_{22}\, \lambda$	$0.1 \times 0.9 \times 0.40 \times 0.1$	0.00360
Θ_7	—	—	$(1-P_D)^2\, \lambda^2$	$0.1^2 \times 0.1^2$	0.00010

Total $= 0.16200 + 0.00405 + 0.00450 + 0.00045 + 0.00090 + 0.00360 + 0.00010 = 0.17560$. Normalize by dividing each weight by 0.17560.

Example Results: The β Weights

Normalised event probabilities: $P(\Theta_1)$=0.92255, $P(\Theta_2)$=0.02306, $P(\Theta_3)$=0.02563, $P(\Theta_4)$=0.00256, $P(\Theta_5)$=0.00513, $P(\Theta_6)$=0.02050, $P(\Theta_7)$=0.00057.

Track 1 (T_1)

$$\beta_{1,1} = P(\Theta_1)+P(\Theta_3) = 0.92255+0.02563 = \mathbf{0.94818}$$
$$\beta_{1,2} = P(\Theta_2)+P(\Theta_4) = 0.02306+0.00256 = \mathbf{0.02563}$$
$$\beta_{1,0} = P(\Theta_5)+P(\Theta_6)+P(\Theta_7) = \mathbf{0.02620}$$

Check: $0.94818 + 0.02563 + 0.02620 \approx 1.000$ ✓

Track 2 (T_2)

$$\beta_{2,1} = P(\Theta_2)+P(\Theta_5) = 0.02306+0.00513 = \mathbf{0.02819}$$
$$\beta_{2,2} = P(\Theta_1)+P(\Theta_6) = 0.92255+0.02050 = \mathbf{0.94305}$$
$$\beta_{2,0} = P(\Theta_3)+P(\Theta_4)+P(\Theta_7) = \mathbf{0.02876}$$

Check: $0.02819 + 0.94305 + 0.02876 \approx 1.000$ ✓

Marginalising for Track 1. To compute each β weight for T_1, scan the "$T_1 \leftarrow$" column and group the events by which observation T_1 receives:

- $\beta_{1,1}$ (Track 1 gets y_1): The "$T_1 \leftarrow$" column contains y_1 in events Θ_1 and Θ_3.
- $\beta_{1,2}$ (Track 1 gets y_2): T_1 receives y_2 in events Θ_2 and Θ_4.
- $\beta_{1,0}$ (Track 1 gets nothing): T_1 is undetected ("—") in events Θ_5, Θ_6, and Θ_7.

The Crucial Difference: PDAF vs JPDA

In PDAF: For a single track i, the weights sum to 1:

$$\sum_{j=0}^{m} \beta_{ij} = 1 \qquad \text{(ignores other tracks entirely)}$$

In JPDA: For a single *observation* j, the sum across all targets is bounded:

$$\sum_{i=1}^{T} \beta_{ij} \;\leq\; 1$$

What this means in practice:

- If observation y_1 fits T_1 very well, then $\beta_{1,1}$ will be large.
- The joint constraint forces $\beta_{2,1}$ to be *correspondingly small* — the probability mass has been "used up" by T_1.
- The result: nearby tracks **push apart** instead of merging.

Key Takeaways

1. **JPDA = PDAF + exclusion constraint.**
   It adds the rule that one observation cannot belong to two targets. Everything else stays the same.

2. **It plugs directly into the Kalman Filter.**
   Standard KF predict $\rightarrow$ JPDA computes β weights $\rightarrow$ PDAF-style update of $\hat{\boldsymbol{\mu}}_{i,\,t|t}$ and $\boldsymbol{\Sigma}_{i,\,t|t}$.

3. **Tracks repel instead of merging.**
   Because joint events split probability mass, nearby tracks naturally push apart rather than coalescing.

4. **Complexity grows with overlap.**
   The number of valid joint events is combinatorial in the number of shared observations. In practice, gating keeps the problem manageable.

Multiple Hypothesis Tracking (MHT): Deferred Decisions

PDAF and JPDA commit at each scan. MHT defers: maintain a tree of all feasible joint-event sequences over *multiple scans*.

Hypothesis score (log-likelihood for track τ):

$$\Lambda(\tau) = \sum_{k \in \mathcal{K}_\tau} \log \frac{P_D \, \mathcal{L}_k}{\lambda} + |\mathcal{K}_\tau^c| \, \log(1 - P_D)$$

where $\mathcal{K}_\tau$ are scans with a detection, $\mathcal{K}_\tau^c$ are misses.

Pruning to control exponential growth:

- N**-scan:** hypotheses differing more than N scans back are merged
- **Score threshold:** delete $\Lambda(\tau) < \epsilon$
- k**-best:** keep only the top-k highest-scoring hypotheses per cluster (Murty)

Connection to RFS

The hypothesis-oriented MHT (HOMHT) is an approximation to the Generalised Labelled Multi-Bernoulli (GLMB) filter — the exact Bayesian multi-target filter in RFS theory. MHT is not ad hoc; it has a rigorous Bayesian foundation.

Block 3 Summary: Method Comparison

	GNN	PDAF	JPDA	MHT
Assignment type	Hard	Soft/per-track	Soft/joint	Deferred
Covariance inflation	No	Yes	Yes	Yes
Inter-target constraint	No	No	Yes	Yes
Closely-spaced targets	Poor	Coalescence	Good	Best
Computational cost	$O(T^3)$	$O(m)$	$O(m^T)$ cluster	$O(2^{NT})$
Track identities	Yes	Yes	Yes	Yes
Variable N	No	No	No	Partial

Design rule

Low clutter, well-separated targets: GNN is fine.
Moderate clutter, one target: PDAF.
Multiple closely-spaced targets: JPDA or MHT.
Need track continuity over many missed detections: MHT.

Block 4

Registration and System Integration

PHD Filter and Complete Multi-Sensor Pipeline

Probability Hypothesis Density (PHD) Filter

PDAF, JPDA, MHT all assume that the number of objects K is known (or approximately constant).
Reality: targets are born and die during the scan interval.

Why the KF framework breaks:

- Fixed state vector $\boldsymbol{x} \in \mathbb{R}^{nK}$ — dimension must change at each birth/death
- Re-initialising the covariance loses information
- No principled model for target birth probability
- Managing track management logic is brittle

What we need:

- A state space that *naturally* accommodates variable cardinality
- A Bayesian filter that is consistent regardless of births and deaths
- A unified treatment of clutter, detection uncertainty, and state estimation

What is the PHD?

- Think of a **heat map of "object-ness"** over the state space (e.g., position/velocity).
- **Brighter areas** mean places we expect to find **more objects**.
- The **sum of brightness** over a region $\Rightarrow$ **expected number of objects** there.
- **Peaks** in the heat map correspond to **likely object locations**.

What is Sensor Registration (and why do we need it)?

- **Problem:** Different sensors rarely agree out of the box:
  - Different **coordinate frames** (mounting angles, boresight).
  - **Systematic biases** (range/angle offsets), **time misalignment**.
  - Measurements in **polar** vs **Cartesian**, with different noise.
- **Registration = Align & debias** all sensors so that the *same object* maps to the *same place* in a common frame and time.
- **If you skip registration:** the same aircraft/car appears shifted for each sensor $\Rightarrow$ the fusion system may create **two tracks for one object** ("ghosts").
- **Intuition:** Before comparing or combining numbers, make sure they are in the *same units, axes, and clocks*, and that fixed offsets are removed.

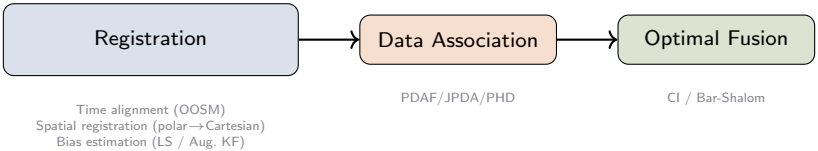
How registration is done (intuitive, no math)

- **Bring to a common frame/time:**
  - Convert each sensor's native coordinates to a shared Cartesian frame.
  - Time-align (account for latency / out-of-sequence arrivals).
- **Estimate & remove biases:**
  - Use normal traffic ("targets of opportunity") to infer small **range/angle** (and other) offsets between sensors.
  - Practical workflow is often **iterative**: *rough association* → estimate biases → correct measurements → *better association*, repeat.
- **Outcome:** After registration, detections/tracks from different sensors *overlay* correctly. Now the fusion/association algorithms see the same reality.

Key separation of roles:

- **Registration** fixes *systematic alignment* (frames, time, biases).
- **Association** decides *which* detection/track corresponds to which object.

The Real Multi-Sensor Pipeline



Each step is a prerequisite for the next. Most real-system failures are failures of registration — not of the association algorithm.

Common engineering mistake

Run a sophisticated JPDA filter on unregistered data. The ghost tracks fool the association algorithm; JPDA tries to track the artefacts. Correct the bias first.

Key insight from all 4 blocks

Block 1, 2 Fusion: *how* to combine?
Block 3 Association: *which* measurements to combine?
Block 4 PHD filter: *how many* targets are there?
Block 4 Regstration: *are we even measuring the same thing?*

Lecture Summary

Core principles:

- **Precision-weighted fusion** (Block 1): Optimal combination of independent Gaussian measurements
- **Information form** (Block 2): Precisions add; missing modalities handled naturally
- **Soft association** (Block 3): PDAF/JPDA capture measurement uncertainty; MHT defers decisions
- **System integration** (Block 4): Registration, then association, then fusion

Unified mathematical view:

- Kalman Filter = recursive Bayesian fusion
- Information filter = precision additivity
- PDAF = probabilistic measurement origin
- JPDA = enforces one-to-one constraint
- Covariance Intersection = conservative fusion under unknown correlation

Practical takeaways:

- *Registration* = make measurements/inputs comparable
- *Association* = decide which things match
- *Fusion* = combine matched, registered information safely

Further Reading

- Bar-Shalom & Fortmann (1988). *Tracking and Data Association*. Academic Press.
- Bar-Shalom & Tse (1975). Tracking in a cluttered environment with probabilistic data association. *Automatica* 11.
- Bar-Shalom et al. (1983). PDAF for target tracking in clutter. *IEEE TAC*.
- Reid (1979). An algorithm for tracking multiple targets. *IEEE TAC* 24.
- Mahler (2003). Multitarget Bayes filtering via first-order multitarget moments. *IEEE TAES* 39.
- Vo & Ma (2006). The Gaussian Mixture PHD filter. *IEEE TSP* 54.
- Mahler (2007). PHD filters of higher order in target number. *IEEE TAES*.
- Schuhmacher et al. (2008). A consistent metric for performance evaluation of multi-object filters. *IEEE TSP*.
- Julier & Uhlmann (1997). A non-divergent estimation algorithm in the presence of unknown correlations. *ACC*.
- Bar-Shalom (2002). Update with out-of-sequence measurements. *IEEE TAES* 38.
- Bar-Shalom & Campo (1986). The effect of common process noise on the two-sensor fused-track covariance. *IEEE TAES*.

Appendix

Mahalanobis Distance

Definition (Scalar and Multivariate)

Mahalanobis distance of a point $\boldsymbol{x} \in \mathbb{R}^d$ from mean $\boldsymbol{\mu}$ under covariance $\boldsymbol{\Sigma} \succ 0$:

$$d_M(\boldsymbol{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) \;=\; \sqrt{(\boldsymbol{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\boldsymbol{x} - \boldsymbol{\mu})}.$$

Squared Mahalanobis distance (often used):

$$D_M^2(\boldsymbol{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) \;=\; (\boldsymbol{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\boldsymbol{x} - \boldsymbol{\mu}).$$

Two-point version (between $\boldsymbol{x}_1$ and $\boldsymbol{x}_2$ w.r.t. $\boldsymbol{\Sigma}$):

$$d_M(\boldsymbol{x}_1, \boldsymbol{x}_2; \boldsymbol{\Sigma}) \;=\; \sqrt{(\boldsymbol{x}_1 - \boldsymbol{x}_2)^\top \boldsymbol{\Sigma}^{-1} (\boldsymbol{x}_1 - \boldsymbol{x}_2)}.$$

Special case: If $\boldsymbol{\Sigma} = \sigma^2$ in $d{=}1$, then $d_M = \frac{|x-\mu|}{\sigma}$ (distance in "sigma units").

Geometric Intuition: Whitening and Ellipses

Whitening view. Let $\boldsymbol{y} = \boldsymbol{\Sigma}^{-1/2}(\boldsymbol{x} - \boldsymbol{\mu})$. Then

$$d_M(\boldsymbol{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) \;=\; \|\boldsymbol{y}\|_2.$$

So Mahalanobis distance is just Euclidean distance *after decorrelating and scaling*.

Elliptical contours. For any $c > 0$,

$$\left\{\boldsymbol{x} : D_M^2(\boldsymbol{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) = c\right\}$$

is an ellipsoid centered at $\boldsymbol{\mu}$ with principal axes given by eigenvectors of $\boldsymbol{\Sigma}$.

Principal axes form. If $\boldsymbol{\Sigma} = Q\Lambda Q^\top$ with $\Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_d)$ and $\boldsymbol{z} = Q^\top(\boldsymbol{x} - \boldsymbol{\mu})$,

$$D_M^2(\boldsymbol{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) \;=\; \sum_{i=1}^{d} \frac{z_i^2}{\lambda_i}.$$

Low-variance directions (small λ_i) are penalized *more*.

Why It Differs from Euclidean Distance

- **Scale-aware:** Features with larger variance contribute less to distance.
- **Correlation-aware:** Redundant movement along correlated features is discounted.
- **Affine invariance:** For invertible A and vector $\boldsymbol{b}$,

$$d_M(A\boldsymbol{x}+\boldsymbol{b}; A\boldsymbol{\mu}+\boldsymbol{b}; A\boldsymbol{\Sigma}A^\top) \;=\; d_M(\boldsymbol{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma}).$$

- **Euclidean as a special case:** If $\boldsymbol{\Sigma} = \boldsymbol{I}$, then $d_M(\boldsymbol{x}; \boldsymbol{\mu}, \boldsymbol{I}) = \|\boldsymbol{x} - \boldsymbol{\mu}\|_2$.

Statistical Interpretation

Gaussian likelihood. For $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, the log-likelihood includes $-\frac{1}{2} D_M^2(\boldsymbol{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma})$.

Chi-square law. If $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, then

$$D_M^2(\boldsymbol{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) \ \sim \ \chi_d^2.$$

This supports:

- **Outlier detection:** Flag $\boldsymbol{x}$ if D_M^2 exceeds a χ_d^2 quantile.
- **Process monitoring:** Hotelling's T^2 statistic in multivariate quality control.
- **Classification:** Appears in LDA/QDA discriminants via precision matrices.

Takeaways

- Mahalanobis distance measures how many *covariance-aware* standard deviations a point is from the mean.
- It equals Euclidean distance in a whitened space, and is invariant to affine transforms.
- For Gaussian data, D_M^2 is χ_d^2 distributed, enabling principled thresholds.
- Reliable use hinges on good covariance estimation (regularize/robustify in practice).

Acknowledgements/ Document preparation

This document was developed and converted into LaTeX slides and formatted with assistance from ChatGPT (OpenAI) and CoPilot (Microsoft). The instructor modified the source text and verified the final structure and wording.